

CS687 Midsem Solutions

Krishna Kumar Bais
Roll No.:241110038

Problem:1

Suppose $f : \Sigma^* \rightarrow \Sigma^*$ is one-to-one (i.e., an injective function) and is partial computable. Show that f^{-1} is partial computable—that is, there exists a Turing machine $M : \Sigma^* \rightarrow \Sigma^*$ such that for every $y \in \text{range}(f)$, $M(y)$ halts and outputs the unique x satisfying $f(x) = y$.

Proof

To show that the inverse function f^{-1} is partial computable given that $f : \Sigma^* \rightarrow \Sigma^*$ is injective and partial computable, we construct a Turing machine M as follows:

- **Turing Machine for f :** Since f is partial computable, there exists a Turing machine M_f that computes f .
- **Algorithm for $f^{-1}(y)$:**
 - **Input:** A string $y \in \Sigma^*$.
 - **Enumeration:** Enumerate all $x \in \Sigma^*$ in a computable order (e.g., lexicographically).
 - **Dovetailing:** Simulate $M_f(x)$ for all enumerated x in parallel by dovetailing their computations.
 - **Output Check:** For each x , if $M_f(x)$ halts and outputs y , then output x and halt.
- **Injectivity Ensures Uniqueness:** Since f is injective, there is at most one x such that $f(x) = y$. Therefore, if $y \in \text{range}(f)$, the correct x will eventually be found.
- **Partial Computability:** If $y \notin \text{range}(f)$, the machine M will never find an x with $f(x) = y$ and will diverge. This behavior is consistent with f^{-1} being a partial function.

Conclusion:

The Turing machine M computes f^{-1} , proving that f^{-1} is partial computable. Thus, the Turing machine M computes the inverse function f^{-1} on its domain, which establishes that f^{-1} is partial computable.

Problem:2

Consider a string x with $K(x) \geq |x| - c$. Let y be an arbitrary string. Show that $|x| + K(y | x) \leq K(x, y) + O(1)$. (In words, if you condition on an incompressible string, the symmetry of information inequality becomes much simpler.)

Proof:

Symmetry of Information

The Symmetry of Information states that:

$$K(x, y) = K(x) + K(y|x, K(x)) + O(1).$$

This equation expresses that the Kolmogorov complexity of the pair (x, y) can be decomposed into:

- The complexity of x , denoted $K(x)$.
- The conditional complexity of y given both x and $K(x)$, written as $K(y|x, K(x))$.
- A small additive constant term $O(1)$.

Apply the Incompressibility Condition

For an incompressible string x , we assume that:

$$K(x) \geq |x| - c,$$

for some small constant c . Since $K(x)$ is at least $|x| - c$, we approximate it by:

$$K(x) \approx |x|$$

up to an additive constant c . This means that the complexity of x is close to its length.

Approximate the Conditional Complexity Term

The term $K(y|x, K(x))$ represents the complexity of y given both x and the value of $K(x)$. Since encoding $K(x)$ takes only $O(1)$ bits, the additional knowledge of $K(x)$ doesn't significantly change the conditional complexity of y . That is:

$$K(y|x, K(x)) = K(y|x) + O(1).$$

This step uses the fact that the presence of $K(x)$ doesn't provide much additional information beyond what x already gives.

Substitute into the Symmetry of Information Equation

We substitute $K(y|x, K(x)) = K(y|x) + O(1)$ into the Symmetry of Information equation:

$$K(x, y) = K(x) + K(y|x, K(x)) + O(1).$$

This simplifies to:

$$K(x, y) = K(x) + K(y|x) + O(1).$$

Using the incompressibility condition $K(x) \geq |x| - c$, we substitute $K(x)$ in:

$$K(x, y) \geq (|x| - c) + K(y|x) + O(1).$$

Rearrange the inequality:

$$|x| + K(y|x) \leq K(x, y) + c + O(1).$$

Since both c and $O(1)$ are constants, we can absorb them into a single $O(1)$ term, giving:

$$|x| + K(y|x) \leq K(x, y) + O(1).$$

Conclusion

Thus, we have proved the desired inequality:

$$|x| + K(y|x) \leq K(x, y) + O(1).$$

This shows that if x is incompressible (i.e., its complexity is close to its length), then conditioning on x simplifies the symmetry of information.

Problem:3

Suppose, for every $k > 1$, you can obtain the first k bits of Chaitin's Omega (described in Question 7 of Homework 1). Using this, define an algorithm to decide whether any program $p \in P$ of length $< k$ halts, where P is the prefix-free set of programs.

Proof:

We present an algorithm to determine whether a given program halts, given access to the first k bits of Chaitin's Omega (Ω_k).

Algorithm

Algorithm 1 Halting Decision for Programs of Length $< k$

```
1: Input: First  $k$  bits of Chaitin's Omega ( $\Omega_k$ )
2: Output: Halting status for all programs  $p$  with  $|p| < k$ 
3: Initialize cumulative sum  $S \leftarrow 0$ 
4: Enumerate all programs  $p$  with  $|p| < k$  in lexicographic order
5: for each program  $p$  do
6:   if  $S + 2^{-|p|} \leq \Omega_k + 2^{-k}$  then
7:     Output:  $p$  halts
8:      $S \leftarrow S + 2^{-|p|}$ 
9:   else
10:    Output:  $p$  does not halt
11:   end if
12: end for
```

Correctness of the Algorithm

Chaitin's Omega (Ω) is defined as:

$$\Omega = \sum_{\substack{p \text{ halts} \\ p \in P}} 2^{-|p|}$$

where P is a prefix-free set of programs. The first k bits Ω_k provide an approximation satisfying:

$$\Omega \in [\Omega_k, \Omega_k + 2^{-k}).$$

Why This Works:

1. **Bounding the Total Contribution:** The sum of the probabilities of halting programs with $|p| < k$ must satisfy:

$$\sum_{\substack{p \text{ halts} \\ |p| < k}} 2^{-|p|} \leq \Omega < \Omega_k + 2^{-k}.$$

This ensures that our approach correctly identifies all programs that halt without exceeding the known bound of Ω_k .

2. **Soundness:**

- If adding the probability of a new program p to S does not exceed $\Omega_k + 2^{-k}$, then p must halt. Otherwise, our approximation of Ω would be too low.
- If adding the probability of p would exceed $\Omega_k + 2^{-k}$, then p cannot halt. If it did, the total halting probability would exceed the upper bound of Ω , contradicting its definition.

Conclusion

Thus, by sequentially checking programs in increasing order of length and summing their contributions, we can correctly determine their halting behavior.

Problem:4

Show that for every pair of strings x and y , we have $C(x, y) \leq C(x) + C(y) + \log C(x) + \log C(y) + O(1)$

Proof:

To show that for every pair of strings x and y , the plain Kolmogorov complexity $C(x, y)$ satisfies:

$$C(x, y) \leq C(x) + C(y) + \log C(x) + \log C(y) + O(1),$$

we construct a program that encodes both x and y using their shortest programs and accounts for their lengths.

- **Shortest Programs:**

- Let p be the shortest program that generates x , so $|p| = C(x)$.
- Let q be the shortest program that generates y , so $|q| = C(y)$.

- **Encoding Strategy:**

- To construct a program that generates the pair (x, y) , we need to combine p and q in a way that the universal Turing machine (UTM) can parse them correctly.
- The challenge is to ensure that the UTM can distinguish between p and q without ambiguity. This requires encoding the lengths of p and q in a self-delimiting manner.

- **Self-Delimiting Lengths:**

- Use a prefix-free encoding for $C(x)$ and $C(y)$. For example:
 1. Encode $C(x)$ as a binary string preceded by $\log C(x)$ 1s and a 0 (unary encoding for the bit-length).
 2. Similarly, encode $C(y)$ in the same way.
- This ensures that the UTM can determine the lengths of p and q without ambiguity.
- The total overhead for encoding $C(x)$ and $C(y)$ is $\log C(x) + \log C(y) + O(1)$, where $O(1)$ accounts for constant factors in the encoding scheme.

- **Combined Program:**

- The combined program for (x, y) consists of:
 1. The prefix-free encoding of $C(x)$,
 2. The prefix-free encoding of $C(y)$,
 3. The program p ,
 4. The program q .
- The UTM reads the lengths of p and q from the prefix-free encodings, then uses this information to parse p and q correctly.

- **Total Length:**

- The total length of the combined program is:

$$|p| + |q| + \log C(x) + \log C(y) + O(1).$$

- Substituting $|p| = C(x)$ and $|q| = C(y)$, we get:

$$C(x, y) \leq C(x) + C(y) + \log C(x) + \log C(y) + O(1).$$

Conclusion:

The inequality $C(x, y) \leq C(x) + C(y) + \log C(x) + \log C(y) + O(1)$ holds because the combined program for (x, y) includes:

- The shortest programs p and q for x and y ,
- The prefix-free encodings of their lengths $C(x)$ and $C(y)$,

- A constant overhead $O(1)$ for the UTM's implementation.

This proof shows that the Kolmogorov complexity of the pair (x, y) is bounded by the sum of the complexities of x and y , plus logarithmic terms to account for the encoding of their lengths.

$$C(x, y) \leq C(x) + C(y) + \log C(x) + \log C(y) + O(1)$$